МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

ЛАБОРАТОРНА РОБОТА №4

з дисципліни «Бази даних»

на тему:

«Аналітичні SQL-запити (OLAP-аналітика)»

Предметна область: CRM-система турнірів з більярдного спорту (Billiards Tournament CRM)

Виконав:
студент групи ІМ-55
Присяжний С.О.

Київ — 2026

1. Мета та завдання лабораторної роботи

Мета роботи: практичне засвоєння методів проектування та виконання складних аналітичних SQL-запитів у режимі Online Analytical Processing (OLAP) у середовищі СУБД PostgreSQL 18.x; формування багаторівневої зведеної звітності, розрахунок агрегованих фінансових та турнірних показників, застосування базових та партиційованих віконних аналітичних функцій (OVER, PARTITION BY), комбінування різних типів реляційних з'єднань (INNER JOIN, LEFT JOIN / Anti-join, CROSS JOIN) та впровадження скалярних і множинних підзапитів у контексті предметної області CRM-системи турнірів з більярдного спорту.

Основні завдання:

1. Агрегаційні та віконні запити: реалізувати агрегацію даних за допомогою функцій COUNT, SUM, AVG, клауз GROUP BY та фільтрації груп HAVING; розробити просту віконну функцію AVG() OVER () для бенчмаркінгу індивідуальних показників спортсменів на фоні глобального пулу, а також партиційовану віконну функцію AVG() OVER (PARTITION BY ...) для розрахунку середньогрупового рейтингу за спортивними званнями без втрати деталізації рядків.

2. Багатотабличні з'єднання: побудувати запити на базі INNER JOIN для консолідації статистики турнірів та клубів, анти-з'єднання на основі LEFT JOIN з предикатом WHERE ... IS NULL для виявлення незіграних більярдних столів, а також декартового добутку CROSS JOIN із суворою нерівністю для автоматизованого формування пар суперників за коловою системою («round-robin»).

3. Скалярні та множинні підзапити: реалізувати підзапити у предикатах WHERE (порівняння рейтингу з середнім значенням), скалярні підзапити у списку вибірки SELECT (динамічний розрахунок дельти відставання від максимуму) та підзапити в клаузі HAVING (відбір клубів із показниками вище середньозважених по всій системі).

4. Верифікація в СУБД PostgreSQL 18: перевірити коректність виконання всіх запитів на фізичній базі даних турнірної системи, зіставити результати з бізнес-логікою та сформувати аналітичні висновки.

2. Опис аналітичних сценаріїв OLAP у більярдній системі

На відміну від транзакційних сценаріїв OLTP, орієнтованих на точкові модифікації (створення карток гравців, оплата внесків, зміна статусу столу), аналітика OLAP (Online Analytical Processing) розв'язує задачі стратегічного планування, оцінки рентабельності клубів та комплексного аудиту спортивних досягнень. У системі турнірного більярду виділено такі ключові напрямки аналітики:

• **Моніторинг та оптимізація інфраструктури:** підрахунок оснащеності клубів столами, виявлення простоюючого обладнання (столи, де ще не проводились матчі), перевірка готовності локацій до турнірів високої категорії.

• **Спортивний скоринг та бенчмаркінг:** оцінка кваліфікаційного рівня гравців щодо глобального середнього рейтингу системи та всередині конкретного спортивного рангу (МСМК, ЗМС), виявлення лідерів турнірного заліку та розрахунок дистанції відставання до лідера.

• **Фінансово-комерційний аудит турнірів:** аналіз розподілу призових фондів за локаціями, виявлення клубів-флагманів, які залучають вищі інвестиції, а також сегментація спортсменів за сумарним виграшем (відбір призерів топ-дивізіону).

• **Календарне планування та жеребкування:** автоматична генерація повної матриці партисипантів для колових етапів турнірів, що гарантує відсутність дублювання зустрічей.

3. Детальний опис та виконання SQL-запитів OLAP

3.1. Агрегаційні та віконні запити (GROUP BY, Window Functions)

Запит 1: Підрахунок кількості більярдних столів у кожному клубі

• Тип запиту: Блок 1: Агрегація (COUNT + GROUP BY)

• Мета: Визначити місткість та оснащеність кожного клубу більярдними столами для вибору локацій

змагань.
• Синтаксичні особливості: Реляційне з'єднання JOIN ... USING (club_id), агрегатна функція COUNT(bt.table_id), групування GROUP BY c.club_id, c.club_name, сортування за спаданням ORDER BY table_count DESC.

```
select
    c.club_name,
    count(bt.table_id) as table_count
from club c
    join billiard_table bt using (club_id)
group by c.club_id, c.club_name
order by table_count desc;
```

Фактичний результат виконання:

club_name	table_count
Більярдний клуб "Піраміда"	3
Brooklyn Billiards	2

Аналітична інтерпретація: Клуб «Піраміда» володіє більшою інфраструктурною базою (3 столи) порівняно з «Brooklyn Billiards» (2 столи), що робить його пріоритетною площадкою для проведення турнірів із великою кількістю учасників.

Запит 2: Порівняння індивідуального рейтингу спортсмена з глобальним середнім (Віконна функція)

• Тип запиту: Блок 1: Проста віконна функція (OVER ())
• Мета: Зіставити кваліфікаційний рейтинг кожного спортсмена з узагальненим середнім значенням за всією базою без втрати деталізації рядків.
• Синтаксичні особливості: Використання простої віконної функції round(avg(p.rating) over (), 2), конкатенація ПІБ оператором ||, фільтрація непустих значень WHERE p.rating IS NOT NULL.

```
select
    u.first_name || ' ' || u.last_name as player_name,
    p.rating,
    round(avg(p.rating) over (), 2)    as global_avg_rating
from player p
    join users u on p.person_id = u.user_id
where p.rating is not null
order by p.rating desc;
```

Фактичний результат виконання:

player_name	rating	global_avg_rating
Дмитро Білозеров	2630	2516.00
Євген Сталєв	2550	2516.00
Олександр Паламар	2500	2516.00
Сергій Крижановський	2450	2516.00
Діана Міронова	2450	2516.00

Аналітична інтерпретація: Застосування віконної клаузи OVER () дає змогу одночасно бачити атомарні дані спортсмена та глобальний бенчмарк (2516.00). Дмитро Білозеров (+114 очок) та Євген Сталєв (+34 очки) формують авангард професійного рейтингу.

Запит 3: Розрахунок середнього рейтингу в розрізі спортивних звань (Партиціонування)

• Тип запиту: Блок 1: Віконна функція з партиціонуванням (PARTITION BY)
• Мета: Оцінити кваліфікаційний зріз гравців всередині їхніх спортивних звань без схлопування та втрати

персональних даних.
• Синтаксичні особливості: Віконна функція з партиціонуванням avg(p.rating) over (partition by p.sports_title), багаторівневе сортування ORDER BY p.sports_title, p.rating DESC.

```
select
    u.first_name || ' ' || u.last_name                          as player_name,
    p.sports_title,
    p.rating,
    round(avg(p.rating) over (partition by p.sports_title), 2) as avg_title_rating
from player p
    join users u on p.person_id = u.user_id
where p.rating is not null
order by p.sports_title, p.rating desc;
```

Фактичний результат виконання:

player_name	sports_title	rating	avg_title_rating
Дмитро Білозеров	INTERNATIONAL_MASTER_OF_SPORTS	2630	2540.00
Сергій Крижановський	INTERNATIONAL_MASTER_OF_SPORTS	2450	2540.00
Євген Сталєв	HONORED_MASTER_OF_SPORTS	2550	2500.00
Олександр Паламар	HONORED_MASTER_OF_SPORTS	2500	2500.00
Діана Міронова	HONORED_MASTER_OF_SPORTS	2450	2500.00

Аналітична інтерпретація: Конструкція PARTITION BY розбиває набір на аналітичні сегменти на льоту: середній рейтинг у групі МСМК дорівнює 2540.00, а в групі ЗМС — 2500.00. При цьому зберігається повна інформація про кожного учасника.

Запит 4: Визначення провідних призерів турнірів із сумарними виплатами понад 5000 грн

• Тип запиту: Блок 1: Агрегація з пост-фільтрацією (SUM + GROUP BY + HAVING)
• Мета: Сформувати список найбільш комерційно успішних більярдистів за накопичувальним турнірним виграшем.
• Синтаксичні особливості: Агрегація SUM(tr.prize_won), групування GROUP BY p.player_id, u.first_name, u.last_name, пост-фільтрація агрегованих груп HAVING sum(tr.prize_won) > 5000.00.

```
select
    u.first_name || ' ' || u.last_name as player_name,
    sum(tr.prize_won)                  as total_prize_won
from player p
    join users u on p.person_id = u.user_id
    join tournament_registration tr using (player_id)
group by p.player_id, u.first_name, u.last_name
having sum(tr.prize_won) > 5000.00
order by total_prize_won desc;
```

Фактичний результат виконання:

player_name	total_prize_won
Дмитро Білозеров	15000.00
Олександр Паламар	9000.00
Сергій Крижановський	6000.00

Аналітична інтерпретація: Клауза HAVING успішно відсікає спортсменів без значних виплат. Лідер рейтингу Дмитро Білозеров заробив 15000.00 грн, що перевищує виграш інших спортсменів разом узятих.

3.2. Багатотабличні з'єднання (JOIN, LEFT JOIN, CROSS JOIN)

Запит 5: Зведена фінансова статистика клубів за проведеними турнірами

• Тип запиту: Блок 2: Внутрішнє з'єднання (INNER JOIN + SUM + COUNT)

• Мета: Порівняти оборот призових фондів та кількість змагань між більярдними клубами для аудиту фінансових потоків.

• Синтаксичні особливості: Багатотабличне об'єднання club JOIN tournament USING (club_id), агрегатні функції COUNT(t.tournament_id) та SUM(t.prize_pool), групування за первинним ключем клубу.

```
select
    c.club_name,
    count(t.tournament_id) as tournament_count,
    sum(t.prize_pool)      as total_prize_pool
from club c
    join tournament t using (club_id)
group by c.club_id, c.club_name
order by total_prize_pool desc;
```

Фактичний результат виконання:

club_name	tournament_count	total_prize_pool
Більярдний клуб "Піраміда"	2	80000.00
Brooklyn Billiards	1	12000.00

Аналітична інтерпретація: Клуб «Піраміда» провів 2 змагання із загальним бюджетом 80000.00 грн проти 1 змагання на 12000.00 грн у «Brooklyn Billiards», акумулюючи понад 86% комерційного обігу турнірів.

Запит 6: Інвентаризація резервного обладнання без проведених матчів (Anti-join)

• Тип запиту: Блок 2: Ліве зовнішнє з'єднання (LEFT JOIN / Anti-Join)

• Мета: Знайти більярдні столи, які жодного разу не були задіяні в турнірних матчах (потенційний резерв або простій).

• Синтаксичні особливості: Комбінація JOIN club та LEFT JOIN match USING (table_id), фільтрація відсутності відповідності WHERE m.match_id IS NULL (класичний реляційний шаблон Anti-join).

```
select
    c.club_name,
    bt.table_number,
    bt.table_type,
    bt.table_status
from billiard_table bt
    join club c using (club_id)
    left join match m using (table_id)
where m.match_id is null
order by c.club_name, bt.table_number;
```

Фактичний результат виконання:

club_name	table_number	table_type	table_status
Brooklyn Billiards	1	PYRAMID_12FT	AVAILABLE
Brooklyn Billiards	2	POOL_8FT	OCCUPIED
Більярдний клуб "Піраміда"	3	SNOOKER_12FT	MAINTENANCE

Аналітична інтерпретація: Знайдено 3 столи, де ще не проходили матчі. Стіл №1 у Brooklyn Billiards повністю вільний (AVAILABLE) і готовий до призначення, тоді як стіл №3 у «Піраміді» знаходиться на техобслуговуванні (MAINTENANCE).

Запит 7: Моделювання турнірної сітки за коловою системою «кожен з кожним» (CROSS JOIN)

• Тип запиту: Блок 2: Декартове перехресне з'єднання (CROSS JOIN)

• Мета: Сформувати сітку можливих поєдинків між усіма спортсменами без дублювання та гри гравця проти самого себе.

• Синтаксичні особливості: Декартове з'єднання таблиці player самої на себе через CROSS JOIN, додаткові об'єднання з users для імен, фільтрація дублікатів та матчів із самим собою WHERE p1.player_id < p2.player_id.

```
select
    p1.player_id                          as player_1_id,
    u1.first_name || ' ' || u1.last_name as player_1_name,
    p2.player_id                          as player_2_id,
    u2.first_name || ' ' || u2.last_name as player_2_name
from player p1
    join users u1 on p1.person_id = u1.user_id
    cross join player p2
    join users u2 on p2.person_id = u2.user_id
where p1.player_id < p2.player_id
order by p1.player_id, p2.player_id;
```

Фактичний результат виконання:

player_1_id	player_1_name	player_2_id	player_2_name
1	Дмитро Білозеров	2	Олександр Паламар
1	Дмитро Білозеров	3	Сергій Крижановський
1	Дмитро Білозеров	4	Євген Сталєв
1	Дмитро Білозеров	5	Діана Міронова
2	Олександр Паламар	3	Сергій Крижановський
2	Олександр Паламар	4	Євген Сталєв
2	Олександр Паламар	5	Діана Міронова
3	Сергій Крижановський	4	Євген Сталєв
3	Сергій Крижановський	5	Діана Міронова
4	Євген Сталєв	5	Діана Міронова

Аналітична інтерпретація: Для N = 5 спортсменів сформовано рівно C(5,2) = 10 унікальних поєдинків. Умова строгої нерівності усуває дзеркальні дублі (наприклад, пара 1-2 та 2-1) та неможливі зустрічі гравця із самим собою.

3.3. Підзапити (Subqueries у WHERE, SELECT, HAVING)

Запит 8: Відбір спортсменів із рейтингом вище середнього (Підзапит у WHERE)

• Тип запиту: Блок 3: Предикатний підзапит у секції WHERE

• Мета: Визначити елітну групу гравців, чий персональний показник строго перевищує загальну норму по всій базі.

• Синтаксичні особливості: Скалярний некорельований підзапит у фільтрі WHERE p.rating > (select avg(rating) from player).

```
select
    u.first_name || ' ' || u.last_name as player_name,
    p.sports_title,
    p.rating
```

```
from player p
    join users u on p.person_id = u.user_id
where p.rating > (
    select avg(rating)
    from player
)
order by p.rating desc;
```

Фактичний результат виконання:

player_name	sports_title	rating
Дмитро Білозеров	INTERNATIONAL_MASTER_OF_SPORTS	2630
Євген Сталєв	HONORED_MASTER_OF_SPORTS	2550

Аналітична інтерпретація: Середній рейтинг гравців бази становить 2516.00. Тільки Дмитро Білозеров та Євген Сталєв долають цей бар'єр, що дає підставу для їхнього сіяння в якості лідерів посіву турнірів.

Запит 9: Розрахунок відставання кожного гравця від поточного лідера (Підзапит у SELECT)

• Тип запиту: Блок 3: Скалярний підзапит у секції SELECT

• Мета: Обчислити аналітичну дельту очок між кожним учасником та першим номером рейтингу турнірної системи.

• Синтаксичні особливості: Використання скалярного підзапиту (select max(rating) from player) безпосередньо в арифметичному виразі списку проекції SELECT.

```
select
    u.first_name || ' ' || u.last_name          as player_name,
    p.rating,
    (select max(rating) from player)            as max_rating,
    (select max(rating) from player) - p.rating as diff_from_max
from player p
    join users u on p.person_id = u.user_id
where p.rating is not null
order by p.rating desc;
```

Фактичний результат виконання:

player_name	rating	max_rating	diff_from_max
Дмитро Білозеров	2630	2630	0
Євген Сталєв	2550	2630	80
Олександр Паламар	2500	2630	130
Сергій Крижановський	2450	2630	180
Діана Міронова	2450	2630	180

Аналітична інтерпретація: Лідером є Дмитро Білозеров з показником 2630 (дистанція 0). Найближчий переслідувач Євген Сталєв відстає на 80 балів, а решта учасників мають гандикап від 130 до 180 балів.

Запит 10: Клуби з середнім призовим фондом вище загальносистемного (Підзапит у HAVING)

• Тип запиту: Блок 3: Фільтраційний підзапит у секції HAVING

• Мета: Виділити клуби преміум-сегменту, чий середній бюджет турніру вищий за середньозважений системний показник.

• Синтаксичні особливості: Порівняння агрегату avg(t.prize_pool) з підзапитом (select avg(prize_pool) from tournament) у клаузі HAVING після реляційного групування.

```
select
    c.club_name,
```

```
    round(avg(t.prize_pool), 2) as avg_club_prize_pool
from club c
    join tournament t using (club_id)
group by c.club_id, c.club_name
having avg(t.prize_pool) > (
    select avg(prize_pool)
    from tournament
)
order by avg_club_prize_pool desc;
```

Фактичний результат виконання:

club_name	avg_club_prize_pool
Більярдний клуб "Піраміда"	40000.00

Аналітична інтерпретація: Загальний середній призовий фонд турнірів становить (50000 + 30000 + 12000) / 3 = 30666.67 грн. Клуб «Піраміда» із середнім показником 40000.00 грн єдиний проходить фільтр, підтверджуючи свій статус головної арени змагань.

4. Порівняльний аналіз OLAP та OLTP, оптимізація аналітичних запитів

1. Навантаження та селективність: OLTP-системи обробляють велику кількість коротких транзакцій із високою селективністю за первинними ключами. На противагу цьому, OLAP-запити сканують великі обсяги історичних рядків, виконують сортування та агрегацію.
2. Ефективність віконних функцій проти підзапитів: традиційне обчислення середнього за категорією через підзапит у SELECT або підсумковий JOIN вимагає повторного сканування таблиці. Віконні функції з клаузою OVER (PARTITION BY ...) обчислюються оптимізатором PostgreSQL в один прохід через внутрішній буфер вікна (WindowAgg), суттєво зменшуючи I/O та час відгуку.
3. Стратегії індексування для OLAP: для прискорення аналітичних операцій рекомендовано створення складених B-Tree індексів на стовпцях зовнішніх ключів та полів фільтрації (наприклад, player(sports_title, rating)), а для важких аналітичних звітів — матеріалізованих представлень (MATERIALIZED VIEW).

5. Зведений SQL-файл запитів (queries.sql)

Повний текст усіх 10 запитів без сторонніх коментарів зі збереженням чистого та уніфікованого синтаксису оформлено у файлі queries.sql у кореневій директорії лабораторної роботи labs/4 - OLAP/queries.sql. Скрипт перевірено на працездатність та узгодженість із фізичною схемою бази даних під керуванням СУБД PostgreSQL 18.

6. Висновки

У ході виконання Лабораторної роботи №4:

1. Освоєно практичні аспекти побудови аналітичних SQL-запитів у режимі Online Analytical Processing (OLAP) на базі реляційної СУБД PostgreSQL 18.x.

2. Реалізовано та протестовано повний спектр аналітичних конструкцій: базові та групові агрегації (COUNT, SUM, AVG, GROUP BY, HAVING), розширені віконні функції з партиціонуванням (OVER, PARTITION BY), реляційні з'єднання (INNER, LEFT / Anti-join, CROSS) та скалярні/множинні підзапити у WHERE, SELECT, HAVING.

3. Проведено порівняльний аналіз архітектур OLTP та OLAP, доведено обчислювальну перевагу віконних аналітичних функцій PostgreSQL перед багаторазовими корельованими підзапитами.

4. Усі 10 розроблених запитів успішно верифіковано на тестовій базі даних CRM-системи більярдних турнірів; результати підтвердили коректність бізнес-моделі та цілісність реляційних даних.